DBMON-01: Database Monitoring Fundamentals

> **Series:** DBMON — Database Monitoring | **Notebook:** 1 of 7 | **Created:** March 2026 | **Last Updated:** 08/11/2026

Overview

This notebook introduces how Dynatrace monitors databases across your environment. You will learn how OneAgent automatically detects database calls through distributed tracing, how database spans capture query-level detail, and how the Dynatrace entity model represents database services. We also cover ActiveGate extensions for remote database monitoring and the full range of supported database technologies.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
OneAgent	Deployed on application hosts making database calls
Permissions	`storage:spans:read`, `storage:entities:read`, `storage:metrics:read`
Data	At least 1 hour of application traffic generating database calls

1. How Database Monitoring Works

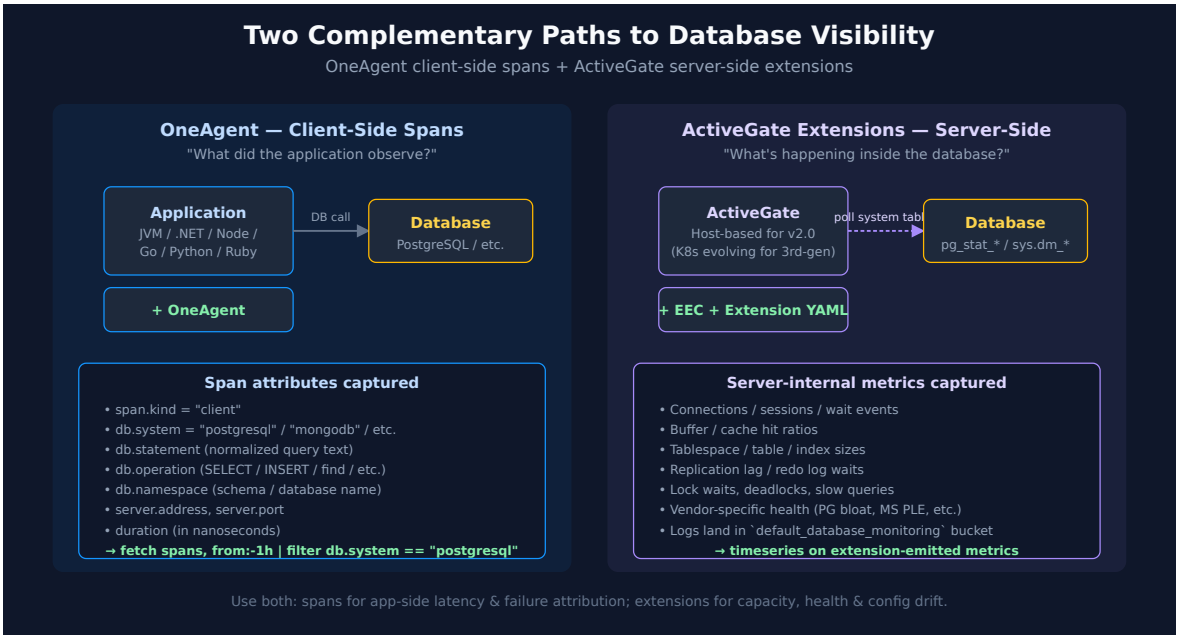
Dynatrace monitors databases through two complementary approaches:

Approach	How It Works	What It Captures
OneAgent Code-Level Instrumentation	Automatically instruments database client libraries (JDBC, ADO.NET, database drivers)	Query text, execution time, result counts, connection details
ActiveGate Extensions	Connects remotely to database management interfaces (SQL queries against system tables)	Internal DB metrics: tablespace usage, lock waits, buffer cache hit ratio

OneAgent captures every database call as a **span** in the distributed trace. These spans contain:

- **`db.system`** — The database technology (e.g., `postgresql`, `mysql`, `mongodb`)
- **`db.statement`** — The query or command executed (normalized to remove literals)
- **`db.operation`** — The operation type (`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `find`, `aggregate`)
- **`db.namespace`** — The database or schema name
- **`server.address`** — The database server hostname or IP
- **`server.port`** — The database server port

> **Note:** Dynatrace normalizes SQL statements by replacing literal values with `?` placeholders. This groups identical query patterns together regardless of parameter values.



2. Database Span Anatomy

Let's examine the structure of a database span by querying for recent database calls and inspecting the available fields.

```
// Inspect database span fields — sample 10 recent DB calls
fetch spans, from:-1h
| filter isNotNull(db.system)
| fields timestamp, db.system, db.operation, db.namespace,
        db.statement, server.address, server.port,
        duration, span.kind, dt.entity.service
| sort timestamp desc
| limit 10
```

The query above returns the key attributes of each database span:

Field	Description	Example
`db.system`	Database technology identifier	`postgresql`, `mysql`, `mongodb`

Field	Description	Example
`db.operation`	SQL command or DB operation	`SELECT`, `INSERT`, `find`
`db.namespace`	Database or schema name	`orders_db`, `inventory`
`db.statement`	Normalized query text	`SELECT * FROM orders WHERE id = ?`
`server.address`	Database host	`db-prod-01.internal`
`duration`	Execution time in nanoseconds	`1500000` (1.5ms)
`span.kind`	Always `CLIENT` for outgoing DB calls	`CLIENT`

3. Discovering Database Services

Dynatrace automatically creates **database service entities** when it detects database calls. These entities represent the logical database endpoint, not the host. Let's discover what database services exist in your environment.

```
// Discover which services call databases, and what they call.
// Note: `serviceType`, `databaseVendor`, `databaseHostNames` and
`softwareTechnologies` are
// CLASSIC dt.entity.service attributes — the Smartscape SERVICE node does not
carry them
// (its model is id / id_classic / name / type / tags / lifetime /
references), so filtering
// a Smartscape node on serviceType == "DATABASE_SERVICE" matches nothing.
Under service
// detection v2 there is no separate DATABASE_SERVICE entity at all: a
database call is a
// CLIENT span on the calling service, described by db.* attributes. Start
from the spans
// and resolve the service identity from there.
fetch spans, from:-24h
| filter isNotNull(db.system)
| summarize call_count = count(), by:{dt.entity.service, db.system,
server.address}
| fieldsAdd calling_service = entityName(dt.entity.service,
type:"dt.entity.service")
| fieldsKeep calling_service, db.system, server.address, call_count
| sort call_count desc
| limit 50
```

You can also discover databases through the spans themselves, which is useful when entity detection hasn't yet completed or when you want to see databases called from specific services.

```
// Discover database technologies from span data
fetch spans, from:-1h
| filter isNotNull(db.system)
| summarize {
    call_count = count(),
    avg_duration_ms = avg(duration) / 1ms,
    distinct_statements = countDistinct(db.statement)
}, by:{db.system, db.namespace, server.address}
| sort call_count desc
| limit 20
```

4. Database Span Exploration

Understanding the distribution of database calls helps identify which databases are most heavily used and where optimization efforts should focus.

```
// Database call volume by technology over the last hour
fetch spans, from:-1h
| filter isNotNull(db.system)
| summarize {
    total_calls = count(),
    avg_duration_ms = avg(duration) / 1ms,
    p95_duration_ms = percentile(duration, 95) / 1ms,
    max_duration_ms = max(duration) / 1ms
}, by:{db.system}
| sort total_calls desc
```

```
// Database call volume over time — identify traffic patterns
fetch spans, from:-6h
| filter isNotNull(db.system)
| makeTimeseries call_count = count(), by:{db.system}, interval:5m
```

```
// Distribution of database operations (SELECT vs INSERT vs UPDATE vs DELETE)
fetch spans, from:-1h
| filter isNotNull(db.system) and isNotNull(db.operation)
| summarize op_count = count(), by:{db.system, db.operation}
| sort db.system asc, op_count desc
```

5. Database Types and Technologies

Dynatrace supports a broad range of database technologies through OneAgent auto-instrumentation.

> **Coverage is per client library, not per database.** "Auto-instrumented" describes the *driver* the application uses, so support arrives library by library and OneAgent version by OneAgent version. Redis on .NET is the clearest example: **StackExchange.Redis** is auto-instrumented from **OneAgent 1.337** and **ServiceStack.Redis** only from **OneAgent 1.343** — the same database, two client libraries, six sprints apart. Where a `db.system` value below is missing from your spans, check the client library and the OneAgent version on the calling host before concluding the database is unmonitored.

Category	Technologies	`db.system` Values
Relational (SQL)	PostgreSQL, MySQL/MariaDB, Microsoft SQL Server, Oracle, IBM Db2	`postgresql`, `mysql`, `mssql`, `oracle`, `db2`
NoSQL Document	MongoDB, Couchbase, Amazon DynamoDB, Azure Cosmos DB	`mongodb`, `couchbase`, `dynamodb`, `cosmosdb`
NoSQL Key-Value	Redis, Memcached, Amazon ElastiCache	`redis`, `memcached`
NoSQL Column	Apache Cassandra, Apache HBase, ScyllaDB	`cassandra`, `hbase`
Search Engines	Elasticsearch, OpenSearch, Apache Solr	`elasticsearch`, `opensearch`, `solr`
Message Brokers	Apache Kafka, RabbitMQ, Amazon SQS	`kafka`, `rabbitmq`
Graph	Neo4j, Amazon Neptune	`neo4j`, `neptune`

> **Important:** The `db.system` field follows the OpenTelemetry semantic conventions. The exact values may vary depending on the database driver and instrumentation version.
>
> **Semantic Dictionary 1.340 (May 2026) — first-class Smartscape database models.** The Semantic Dictionary now defines dedicated Smartscape models for major database technologies: Oracle Database (ASM disk group, cluster, instance, database), SQL Server (availability database / group / replica, instance, database), SAP HANA (database, instance, service), IBM Db2 (database member, instance, tablespace), MySQL, MariaDB, and PostgreSQL (database + instance each) — **21 models** in total, each with `belongs_to` / `runs_on` / `is_part_of` / `same_as` / `uses` relationship properties. (Verified against the live Semantic Dictionary: `fetch dt.semantic_dictionary.models | filter startsWith(name, "dt.smartscape.db_")` returns 21.) As these roll out, expect database topology to surface as typed Smartscape nodes rather than only the generic `DATABASE_SERVICE` shape used in §3. The span-level `db.system` analysis in this series is unaffected.
>
> **OneAgent 1.339 (June 2026) — Db2 naming is now mixed case.** Db2 database naming switched to mixed case, so saved DQL or filters that match Db2 *entity or database names* case-sensitively may need updating. The `db.system == "db2"`

> span-attribute filters used throughout this series follow the OpenTelemetry semantic convention (lowercase value) and are unaffected.

```
// Discover which database technologies are active in your environment
fetch spans, from:-24h
| filter isNotNull(db.system)
| summarize {
    call_count = count(),
    unique_databases = countDistinct(db.namespace),
    unique_servers = countDistinct(server.address)
}, by:{db.system}
| sort call_count desc
```

6. ActiveGate Extensions for Remote Monitoring

While OneAgent captures database calls from the application side (client spans), ActiveGate extensions monitor the database server itself. This provides internal metrics that are invisible from the application perspective.

> **Recommendation for new customers:** build on **Extensions 2.0** — the current extensions framework. Extensions Framework 1.0 reached end of support on 2025-03-31 (Python EF1.0: 2024-10-31); JMX and PMI EF1.0 are deprecated but supported past that date on request. Note that recent Dynatrace doc pages say simply "Extensions" and mean Extensions 2.0.

Extension Architecture

Component	Role
ActiveGate	Hosts the extension runtime; must be **host-based** (not Kubernetes-based) for Extensions 2.0 — verify current Kubernetes-based ActiveGate support against the Dynatrace docs

Component	Role
Extension Package	YAML-defined declarative package executed by the Extension Execution Controller (EEC). Most database extensions ship with built-in SQL/Prometheus/SNMP data sources; custom logic can be added via an optional Python data source
Monitoring Configuration	Defines connection string, credentials, polling interval, and the destination Grail bucket
`default_database_monitoring` bucket	Where extension logs land by default; reference this bucket in IAM policies for DB-team access scoping (see ORGNZ-02 + IAM-04/05)

Available Extensions

Extension	Metrics Captured
PostgreSQL	Connections, transactions/sec, tuple operations, table/index sizes, lock waits, replication lag
MySQL	Connections, queries/sec, buffer pool hit ratio, InnoDB row operations, slow queries
MS SQL Server	Batch requests/sec, page life expectancy, buffer cache hit ratio, deadlocks, wait stats
Oracle	Sessions, tablespace usage, SGA/PGA memory, redo log waits, library cache hit ratio
MongoDB	Connections, operations/sec, document metrics, replica set health, storage engine stats

> **Note:** Extensions 2.0 require a **host-based ActiveGate**, not a Kubernetes-based deployment. See the Dynatrace documentation for extension installation procedures.

Where to Go Deeper

- **AUTOM series** (11 notebooks) — GitOps / Monaco / Terraform automation for extension deployment at scale
- **ORGNZ-02** — Bucket strategy (including `default_database_monitoring`)

- **IAM-04 / IAM-05** — Policy design that scopes extension log access by bucket

Where Extension Logs Land in Grail

Logs emitted by official Dynatrace database extensions are routed to a dedicated Grail bucket: `default_database_monitoring` (introduced in Dynatrace SaaS 1.337). Querying this bucket directly improves filter performance vs. scanning `default_logs`, and lets you scope IAM policies tightly to database-monitoring data.

```
fetch logs, from:-1h
| filter dt.system.bucket == "default_database_monitoring"
| filter dt.extension.name == "com.dynatrace.extension.postgresql"
| limit 50
```

Grant `storage:bucket.default_database_monitoring:read` to roles that need to query this bucket.

Dynatrace Database App — Analysis-First Observability

Beyond OneAgent spans and ActiveGate extension metrics, Dynatrace ships a dedicated **Databases app** that layers automated analysis on top of both data sources. As of August 2026 it is generally available for **PostgreSQL and MySQL**, with additional database technologies planned.

The app evaluates each monitored database instance across five areas — configuration, schema, query, execution plan, and a rolled-up **Health Score** (0–100) — and surfaces AI-generated remediation suggestions backed by Dynatrace Intelligence.

Capability	What it adds beyond this notebook's DQL approach
Health Score	Single 0–100 rating per database instance for triage before deep analysis
Query & execution-plan insights	Interactive execution-plan visualization, normalized across PostgreSQL, MySQL, and SQL Server
Configuration insights	Flags parameter mismatches against recommended settings
Schema insights	Structural warning signals (missing indexes, oversized tables)
AI-powered remediation	Suggested fix with reasoning, generated by Dynatrace Intelligence

Treat the app as a triage front end, not a replacement for the span- and extension-level analysis in this series — the DQL patterns in DBMON-01/05/06 remain the way to build the custom dashboards, alerts, and SLOs the app's UI doesn't cover.

> **Sources:** Introducing Intelligent Database Observability (DT blog), Databases app (DT docs).

7. Baseline Database Metrics

Establishing a performance baseline is essential for detecting anomalies. The following queries help you understand your typical database performance characteristics.

```
// Database response time baseline — hourly P50, P95, P99 over the last 24
hours
fetch spans, from:-24h
| filter isNotNull(db.system)
| makeTimeseries p50_ms = percentile(duration / 1ms, 50),
                 p95_ms = percentile(duration / 1ms, 95),
                 p99_ms = percentile(duration / 1ms, 99),
                 interval:1h
```

```
// Error rate baseline — database call failures over time
fetch spans, from:-24h
| filter isNotNull(db.system)
| makeTimeseries total = count(),
                 errors = countIf(span.status_code == "error", default:0),
                 interval:1h
| fieldsAdd error_rate_pct = round(arraySum(errors) / arraySum(total) * 100,
decimals:2)
```

8. Summary and Next Steps

In this notebook you learned:

- How Dynatrace captures database activity through OneAgent span instrumentation
- The key span attributes (`db.system`, `db.statement`, `db.operation`, `db.namespace`) that describe each database call
- How to discover database service entities and active database technologies

- The role of ActiveGate extensions for server-side database metrics
- How to establish a performance baseline using span data

Next Steps

- **DBMON-02: SQL Database Monitoring** — Deep dive into relational database analysis with PostgreSQL, MySQL, MS SQL, and Oracle
- **DBMON-03: NoSQL Database Monitoring** — MongoDB, Cassandra, DynamoDB, and Cosmos DB analysis

---